

Bisou App

Das umfassende Handbuch & Technische Systemdokumentation

Version: 9.0 (Mobil-Optimiert) • Datum: 20. Juni 2026 • Entwickler: Benedikt S.

System-Inhaltsverzeichnis

Abschnitt / Kapitel	Beschreibung der App-Komponenten	Seite
1. Ausführliches Benutzerhandbuch (Laien)	Ausführliche Erklärung aller Alltags-Funktionen (Fragen, Spionschutz, Streaks, Freeze, Score).	1
2. Das Fragensystem & KI-Generierung	Struktur der daily_questions, Gemini-Prompts und failed_generations Retry-Queue.	2
3. Beantwortungsprozess & Serialisierung	Wizard-Schritte, LocalStorage-Zustandssicherung und Signatur-Vergleich.	3
4. Der Kompatibilitäts-Algorithmus	Kosinus-Ähnlichkeit (gte-small), Spearman-Rangabstand und dynamische Gewichtung.	4
5. Streak- & Freeze-Zustandsmaschine	check_and_freeze_streak, automatische Freezes und Erhalt der streak_history.	5
6. Erfolge- & Meilenstein-Zählersystem	Die 20+ persistenten Zähler, Feiertags-Flags und clientseitige Deduplizierung.	6
7. Server-Driven UI & Ankündigungen	Ankündigungen-Parser (renderAnnouncementContent) für Listen und Textzeilen.	7
8. Frontend-Architektur & Tab-Sync	Virtual-Canvas responsive Skalierung, SafeAuthChannel (BroadcastChannel) für iOS.	7
9. Designkonzept & Micro-Animations	HSL-Farbvariablen, Fraunces/Jakarta Sans, Keyframe-Effekte (flicker, wobble).	8
10. Datenbankschema & RLS Policies	PostgreSQL DDL-Matrix (profiles, answers, streaks, failed_generations, announcements).	9
11. PL/pgSQL Funktionen & RPCs	Partnerkopplung link_partners, unlink_partners, Account-Löschung, Antwort-Reset.	10
12. Web-Push & Deno Edge Functions	Kryptografische PWA Push-Zustellung (ES256 VAPID, ECDH, AES-128-GCM).	11
13. Systemdatenfluss & Datenarchitektur	Visuelle Datenmatrix vom Client über Trigger bis zur Edge-Function.	12

1. Ausführliches Benutzerhandbuch: Funktionsweise der Bisou App (Laien-Erklärung)

Die Bisou App ist eine private PWA (Progressive Web App) für Paare, die ein tägliches Ritual der Nähe und des Austauschs pflegen wollen. Die Bedienung ist denkbar einfach und auf Langlebigkeit ausgelegt:

- **Das tägliche Ritual:** Jeden Tag um 12:00 Uhr mittags stellt die App vier neue Fragen bereit. Diese decken unterschiedliche emotionale Dimensionen ab: *Dies oder Das* (Entscheidung), *Prioritäten-Ranking* (Sortierung), *Freitext* (offene Frage zum Tippen) und *Wer würde eher* (wwe - spielerischer Partnervergleich).
- **Der Spionenschutz (Abschreibeschutz):** Damit eure Antworten ehrlich und unbeeinflusst bleiben, sind die Antworten des Partners für den heutigen Tag mit einem Schloss gesperrt. Erst wenn du deine eigenen Fragen vollständig beantwortet hast, wird die Auswertung freigeschaltet und du siehst die Gedanken deines Partners.
- **Die Antwortserie (Streak) & Streak-Freeze:** Für jeden Tag, an dem ihr beide antwortet, erhöht sich euer Streak (die Flamme). Vergesst ihr einmal zu antworten, erlischt der Streak normalerweise. Die App besitzt jedoch ein automatisches **Streak-Freeze-System**. Wenn ihr einen Tag verpasst, wird die Serie eingefroren (maximal zweimal im Monat), sodass eure Flamme beim nächsten Antworten nicht erlischt.
- **Der Kompatibilitäts-Score (Bisou Score):** Eure Antworten der letzten 30 Tage werden miteinander abgeglichen. Daraus wird täglich ein Score von 0.0 bis 10.0 berechnet, der eure Ähnlichkeit und Harmonie widerspiegelt. Ein Trendpfeil zeigt an, ob euer Score steigt oder sinkt.
- **Meilensteine & Trophäen:** Besondere Ereignisse (z. B. das erste Mal anstupsen, eine 7-Tage-Serie oder das Beantworten am Valentinstag oder zu Weihnachten) schalten Meilensteine frei. Das Dashboard feiert dies mit einem großen Konfetti-Regen. Alle Erfolge sind in eurem Profil im Popup 'Erfolge' golden markiert.
- **Tagebuch:** Im Tagebuch könnt ihr bis zu 60 Tage zurückblättern, um alte Fragen und eure damaligen Antworten noch einmal zu lesen.

2. Das Fragensystem & die Gemini-KI-Generierung

Die tägliche Interaktion basiert auf vier verschiedenen Fragentypen. Die Bereitstellung der Fragen erfolgt vollautomatisch über eine Supabase Edge Function (generate-questions), die über einen täglichen Cron-Job (mittels pg_cron auf der Datenbank) um 12:00 Uhr mittags getriggert wird.

2.1 JSON-Struktur der täglichen Fragen

In der Tabelle daily_questions werden die Fragen im Feld questions als strukturiertes JSONB-Dokument gespeichert. Ein valides JSON-Dokument besitzt folgendes Schema:

```
{
  "tot": {
    "q": "Was ist euch an einem Sonntag wichtiger?",
    "o": ["Gemütliches Ausschlafen & Frühstück im Bett", "Früh aufstehen & zusammen etwas erleben"]
  },
  "ranking": {
    "q": "Sortiert diese Beziehungsfaktoren nach eurer persönlichen Priorität:",
    "o": ["Gemeinsame Hobbys", "Offene Kommunikation", "Körperliche Nähe", "Gegenseitiger Freiraum"]
  },
  "text": {
    "q": "Was war ein kleiner Moment in dieser Woche, für den du deinem Partner dankbar bist?"
  },
  "wwe": {
    "q": "Wer von euch beiden verliert bei einem Brettspiel eher die Geduld?"
  }
}
```

2.2 Der Prompt-Algorithmus der Gemini API

Falls für ein angefordertes Datum (day_key) noch kein Fragen-Datensatz existiert, formuliert die Edge Function einen detaillierten System-Prompt an das Modell gemini-1.5-flash. Der Prompt instruiert die KI wie folgt:

- Generiere genau 4 Fragen mit den Typen tot (Dies oder Das mit 2 Optionen), ranking (mit exakt 4 Optionen), text (offene Freitextfrage) und wwe (Wer würde eher).
- Die Fragen müssen sich auf die Themen Partnerschaft, Alltag, Träume und gemeinsame Zukunft beziehen.

Erzwinge die Ausgabe als reines, valides JSON-Dokument ohne Markdown-Formatting (keine Backticks wie ``json`), damit der Deno-Server die Antwort direkt in die Datenbank schreiben kann.

2.3 Robuste Fallbacks & failed_generations Retry-Queue

Um einen Systemausfall bei API-Drosselungen oder Netzwerkstörungen der Google API zu verhindern, implementiert die Funktion ein mehrstufiges Sicherheitsnetz:

1. **Lokale Fallback-Datenbank:** Im Code der Edge Function ist eine statische Konstante FALLBACK_QUESTIONS mit vordefinierten Beispielfragen hinterlegt. Schlägt der Gemini-Aufruf fehl, greift das System auf diese zurück, sodass der Benutzer keine Fehlermeldung sieht.
2. **Wiederholungs-Warteschlange (Retry-Queue):** Schlägt die Generierung fehl, schreibt die Funktion einen Datensatz in die Tabelle failed_generations. Ein Datenbank-Cronjob (retry_failed_generations) prüft diese Tabelle stündlich und versucht, die fehlenden Fragen asynchron im Hintergrund erneut per KI zu generieren. Ist dies erfolgreich, wird der Queue-Eintrag gelöscht.

3. Der Beantwortungsprozess & die Datenserialisierung

Der Beantwortungsprozess im Client wird über einen Wizard gesteuert. Die App stellt sicher, dass Teilantworten lokal geschützt sind und dass die Antworten beider Partner sicher miteinander verglichen werden können.

3.1 Clientseitige Zustandssicherung (LocalStorage-Cache)

Während der Benutzer die Fragen beantwortet, speichert die Komponente Questions.tsx nach jedem Einzelschritt den aktuellen Zustand im LocalStorage (Schlüssel: quiz_progress_{dayKey}). Das Objekt speichert den aktuellen Index des Schritts (0 bis 3) und ein Array mit den bisherigen Antworten. Dadurch wird verhindert, dass der Benutzer bei einem Verbindungsabbruch seinen Fortschritt verliert. Sobald der letzte Schritt vollendet und die Antworten erfolgreich an die Supabase-Datenbank übermittelt wurden, wird dieser LocalStorage-Eintrag bereinigt.

3.2 Das serielle Antwort-String-Format & Signaturen

Die Antworten eines Benutzers für einen Tag werden nicht als separate Spalten oder Zeilen abgelegt, sondern als ein einziger, strukturiert serialisierter Text-String in der Spalte choice der Tabelle answers gespeichert. Das Format ist wie folgt aufgebaut:

Antwort1 | Antwort2 | Antwort3 | Antwort4 [Frage1][Frage2][Frage3][Frage4]

Beispiel für einen echten Datenbank-Eintrag:

Sofa & Netflix | Offene Kommunikation > Körperliche Nähe > Hobbys | Zusammen kochen | Partner [tot:Was macht ihr sonntags?][ranking:Sortiert nach Prio...][text:Lieblingsgericht?][wwe:Wer ist ungeduldiger?]

Der Zweck der Signatur-Klammern (Spionschutz & Integrität):

Die am Ende angehängten Signatur-Klammern [...] enthalten den exakten Fragetext zum Zeitpunkt der Beantwortung. Da die Fragen theoretisch im Laufe des Tages regeneriert werden könnten, dient die Signatur als Validierung. Die Auswertungs-Function calculate-stats vergleicht die Signaturen beider Partner. Stimmen die Signaturen nicht überein, erkennt das System, dass die Partner auf unterschiedliche Fragen geantwortet haben. Dies verhindert Berechnungsfehler und falsche Auswertungen bei asynchronen Datumswechseln.

3.3 Ablauf beim Absenden der Daten

Sobald der Benutzer den letzten Schritt abschließt, führt die App eine Datenbanktransaktion aus:

1. Löscht eventuell bereits vorhandene temporäre Antworten des Benutzers für diesen Tag (falls korrigiert wird).
2. Fügt den neu generierten Antwort-String in public.answers ein.
3. Triggert über die Edge-Function eine Push-Benachrichtigung an den Partner, dass die Antworten vorliegen.
4. Startet die lokale clientseitige Verschlüsselungs-Animation und ruft onComplete() auf, um das Dashboard zu aktualisieren.

4. Der Kompatibilitäts- & Matching-Algorithmus (calculate-stats)

Die Berechnung der Übereinstimmungswerte erfolgt in der Deno Edge Function calculate-stats. Sie analysiert die letzten 30 Tage der Beziehung und wendet für jeden Fragentyp spezifische mathematische Verfahren an.

4.1 Die vier Abgleich-Methoden im Detail

- **Dies oder Das (tot) - Binär-Matching:** Ein einfacher String-Vergleich. Stimmen die ausgewählten Optionen überein, beträgt der Match-Wert für diesen Tag 100%, andernfalls 0%.
- **Ranking-Frage - Spearman-Distanz mit Wurzel-Dämpfung:** Die Partner sortieren 4 Optionen. Die Abweichung wird über die Summe der quadrierten Differenzen der Ränge (1 bis 4) berechnet. Der maximal mögliche quadratische Abstand bei 4 Elementen beträgt $d^2_{\text{max}} = 20$. Um eine Überstrafung kleiner Abweichungen zu verhindern und Paare psychologisch zu ermutigen, wird die Ähnlichkeit über eine Wurzelfunktion geglättet:
$$\text{Score} = \sqrt{1 - (\text{Summe}(\text{Rang}_A - \text{Rang}_B)^2 / 20)} * 100$$
- **Freitext-Frage - Semantische Kosinus-Ähnlichkeit:** Die Texte werden durch das in der Edge Function ausgeführte Embedding-Modell gte-small in 384-dimensionale Vektor-Arrays konvertiert. Die mathematische Ähnlichkeit ist das Skalarprodukt der normierten Vektoren:
$$\text{CosineSimilarity} = (A \cdot B) / (||A|| * ||B||)$$

Das System mappt den Bereich [0.3; 0.9] linear auf [0%; 100%]. Schlägt die Vektor-Erstellung fehl, berechnet das System als Fallback den Jaccard-Koeffizienten (Schnittmenge der Wörter geteilt durch die Vereinigungsmenge).
- **Wer würde eher (wwe) - Komplementär-Abgleich:** Da sich Partner bei WWE-Fragen einig sind, wenn einer 'Ich' und der andere 'Partner' wählt, liegt ein Match vor, wenn die Antwort-Strings *ungleich* sind: $\text{choice}_A \neq \text{choice}_B$ (Match: 100%, sonst 0%).

4.2 Dynamische Gewichtung & Berechnung des Bisou-Scores

Der Gesamt-Bisou-Score liegt auf einer Skala von 0.0 bis 10.0. Jede der vier Fragenkategorien hat das gleiche Gewicht von 25% (Faktor 0.25). Um verfälschte Ergebnisse bei unvollständigen Daten zu vermeiden, wendet die Funktion eine **dynamische Gewichtungs-Normalisierung** an:

- Es werden nur Kategorien in die Berechnung einbezogen, bei denen für die betrachteten Tage tatsächlich Antworten beider Partner vorliegen.
- Die Summe der Übereinstimmungsprozente der aktiven Kategorien wird durch die Anzahl der tatsächlich aktiven Kategorien geteilt.
- Der resultierende Prozentwert wird durch 10 geteilt und mathematisch auf eine Dezimalstelle gerundet (z.B. 84.3% Übereinstimmung = Score 8.4).

4.3 Trendberechnung und Trendpfeile

Das Dashboard zeigt neben dem Bisou-Score einen Trendpfeil (Up/Down) an. Um diesen zu ermitteln, berechnet die Edge Function zwei Werte parallel: Den aktuellen Bisou-Score der letzten 30 Tage (inklusive heute) und den vorherigen Bisou-Score (indem der allerneueste gemeinsame Antworttag aus der Berechnung ausgeschlossen wird). Ist der aktuelle Score höher als der vorherige, wird ein steigender Trend signalisiert, andernfalls ein fallender.

5. Streak-Verwaltung & Streak-Freeze System

Das Halten einer Antwortserie (Streak) ist ein wesentlicher Motivationsfaktor der App. Um unverschuldete Serienverluste abzufedern, implementiert die App eine ausgeklügelte Zustandsmaschine zur automatischen Streak-Einfrierung.

5.1 Funktionsweise von `check_and_freeze_streak()`

Jedes Mal, wenn ein Benutzer die App öffnet, triggert der Client die RPC-Datenbankfunktion `check_and_freeze_streak(p_today)`. Die Funktion arbeitet wie folgt:

1. Sie prüft, ob der Streak des Nutzers aktiv ist. Ist das letzte Antwortdatum (`last_answer_date`) heute oder gestern, bleibt der Streak unberührt.
2. Hat der Nutzer gestern nicht geantwortet, ermittelt das System, ob ein **Streak-Freeze** angewendet werden kann. Dazu analysiert die Funktion das Array `freeze_history`. Sie zählt die verbrauchten Freezes im Kalendermonat des Vortages. Liegt die Anzahl bei **unter 2**, wird das gestrige Datum in die `freeze_history` eingetragen und der Streak bleibt erhalten (eingefroren).
3. Sind bereits 2 Freezes für diesen Monat verbraucht, bricht der Streak ab und wird auf 0 zurückgesetzt.

5.2 Integration in `update_streak()` & Erhalt der Streak-Historie

Der AFTER INSERT-Trigger `update_streak` auf Antworten integriert diese Logik nahtlos. Gibt ein Benutzer eine Antwort ab, wird geprüft, ob er gestern verpasst hat. Ist dies der Fall und ein Freeze steht zur Verfügung, wird der Streak fortgeführt (+2 Tage addiert für den Freeze-Tag und heute) und das verpasste Datum in die `freeze_history` eingetragen. **Wichtiger Erhalt der Historie:** Bricht der Streak ab oder wird er zurückgesetzt (oder korrigiert der Nutzer seine Antworten), überschreibt die Funktion `streak_history` nicht mehr mit dem heutigen Tag, sondern hängt das Datum an. Dies verhindert Datenverluste und sichert die Validität der historischen Antwortstatistiken.

6. Erfolge & Meilenstein-Zählersystem

Das Achievement-System der Bisou App ist vollständig entkoppelt von der temporären Tabelle `answers` (die nach 90 Tagen bereinigt wird). Es basiert auf persistenten Zählerfeldern direkt in der Tabelle profiles, die synchron bei Antwort-Interaktionen gewartet werden.

6.1 Die synchronisierten Datenbank-Metriken

Die folgenden 20+ persistenten Metriken werden in public.profiles gepflegt:

- `total_answers`: Erfasst alle jemals eingereichten Antworten des Benutzers.
- `total_matches`: Zählt die Übereinstimmungen aller Dies-oder-Das- und Ranking-Antworten.
- `perfect_tot_days_count`: Zählt Tage mit perfekter Übereinstimmung bei Dies-oder-Das (tot) Fragen.
- `perfect_match_days_count`: Tage, an denen alle Auswahlfragen (tot & wwe) absolut identisch beantwortet wurden.
- `perfect_rankings_count`: Erzielt durch 100%-Übereinstimmungen im Prioritäten-Ranking.
- `time_sync_5min_count` / `time_sync_1min_count`: Erfasst, wie oft Partner innerhalb von 5 Minuten bzw. 60 Sekunden nacheinander geantwortet haben.
- `morning_answers_count` / `night_answers_count` / `lunch_answers_count` / `last_minute_answers_count`: Zeitbasierte Beantwortungsmuster.
- `long_answers_count`: Freitext-Antworten mit mehr als 150 Zeichen.
- `both_long_answers_count`: Tage, an denen beide Partner eine lange Freitext-Antwort (> 200 Zeichen) abgegeben haben.
- `avatar_change_count`: Zählt Profilbild-Aktualisierungen.
- `nudges_sent`: Gesamtzahl aller an den Partner übermittelten Anstupser (nudge).
- `journal_views`: Erfasst das Interesse der Partner am Archiv.
- `streaks_rebuilt`: Zählt, wie oft ein abgebrochener Streak wieder auf 7 Tage aufgebaut wurde.

6.2 Feiertags-Trigger & Meilenstein-Deduplizierung

- **Feiertags-Trigger (Boolean-Flags)**: Die Tabelle profiles hält dedizierte Flags für Feiertage: `answered_valentines` (14. Feb), `answered_new_years` (1. Jan), `answered_christmas` (24.-26. Dez), `answered_halloween` (31. Okt), `answered_labor_day` (1. Mai), `answered_unity_day` (3. Okt), `answered_leap_day` (29. Feb) und `answered_anniversary` (am Jahrestag der Kopplung). Diese Trigger schalten die speziellen Event-Meilensteine frei.
- **Clientseitige Toast-Deduplizierung**: Um wiederholtes Anzeigen desselben Meilenstein-Banners zu unterbinden, liest der Client die Liste bereits gesehener Meilensteine beim Start in ein lokales Set (`seen_milestones_{userId}`) ein. Nur wenn ein neu eingelesener Erfolg nicht in diesem Set existiert, wird der Dashboard-Toast gerendert, Konfetti ausgelöst und die ID dem Set hinzugefügt. Dies entlastet das Backend von wiederholten Sichtbarkeits-Updates.

7. Server-Driven UI & Systemankündigungen

Das Ankündigungssystem ermöglicht es, Benachrichtigungen mit strukturiertem Inhalt direkt aus der Datenbank zu steuern. Dazu wertet die Client-seitige Funktion `renderAnnouncementContent(text)` in `App.tsx` das Textfeld `content` der Tabelle `announcements` zeilenweise aus:

- **Listenpunkte (Bullet-Points):** Zeilen, die mit einem der Sonderzeichen `•`, `-` oder `*` beginnen, werden automatisch erkannt. Das System entfernt das Sonderzeichen sowie führende Leerzeichen und gruppiert alle direkt aufeinanderfolgenden Listenpunkte in eine linksbündige ungeordnete HTML-Liste (`<ul>` und `<li>`) mit Standard-Aufzählungspunkten.
- **Standard-Text:** Normale Textzeilen werden als zentrierte Textabsätze (`<p>`) dargestellt.
- **Absätze & Zeilenumbrüche:** Leere Zeilen im Text erzeugen einen kleinen vertikalen Abstand zur visuellen Gliederung.

8. Frontend-Architektur, Tab-Sync & Dialog-System

Die Client-Architektur fokussiert sich auf eine immersive PWA-Erfahrung, die auch unter restriktiven Bedingungen stabil läuft.

8.1 Der SafeAuthChannel für resilienten Tab-Sync

Um An- und Abmeldungen über mehrere offene Browser-Tabs hinweg synchron zu halten, verwendet die App normalerweise die native BroadcastChannel-API. In restriktiven Umgebungen (wie iOS In-App-Browsern in Instagram, Telegram oder WebViews) kann die Initialisierung der BroadcastChannel-API jedoch zu harten JavaScript-Abstürzen führen. Die App kapselt den Aufruf daher in der Klasse SafeAuthChannel. Tritt beim Erstellen des Kanals oder Senden von Nachrichten ein Fehler auf, wird dieser abgefangen, protokolliert und ein stummer Fallback aktiviert. Dadurch bleibt die App auf allen Endgeräten voll einsatzbereit.

8.2 Responsive UI-Skalierung (Virtual-Canvas)

Die gesamte Anwendung nutzt einen ScalingContainer. Dieser verhält sich wie eine virtuelle Canvas, die das Layout bei extremen Auflösungen oder Seitenverhältnissen (wie iPhone SE oder großen Tablets) proportional skaliert. Dadurch wird das Abschnippeln von Steuerungselementen oder der Dock-Navigationsleiste am unteren Bildschirmrand effektiv verhindert.

8.3 Das anpassbare DialogProvider-System

Die klassischen JavaScript-Funktionen alert() und confirm() wurden vollständig durch React-Modals ersetzt:

- showAlert(message, type): Blendet ein unaufdringliches, aber unübersehbares Overlay mit dem Nachrichtentext ein.
- showConfirm(message, onConfirm, options): Rendert eine Ja/Nein-Abfrage (z. B. beim Zurücksetzen von Antworten) mit anpassbaren Button-Labels.

9. Designkonzept, HSL-Theme & Micro-Animations

Das visuelle Design der Bisou App zeichnet sich durch weiche Verläufe, harmonische Farbtöne und intuitive Bedienung aus.

9.1 HSL-Theme Farbvariablen

Das CSS-Theme steuert alle Farben über native HSL-Variablen, um eine nahtlose Umschaltung zwischen Hell- und Dunkelmodus zu ermöglichen. Die Primärfarbe (Rose/Rot) vermittelt Wärme, während die Sekundärfarbe (Soft Lila) für Vertrauen steht.

CSS Variable	Light Theme	Dark Theme	Semantischer Nutzen
--primary	#FF4757 (Vibrant Red)	#FF4757 (Vibrant Red)	Streaks, Buttons, Akzente
--secondary	#5F27CD (Vibrant Purple)	#B4AFFF (Hell-Lila)	Buttons, Meilensteine
--bg	#F1F2F6 (Cool-White)	#0C0A15 (Tief-Schwarz)	Hintergrundfarbe
--text-main	#2F3542 (Dark Slate)	#F5F3FF (Off-White)	Überschriften und Fließtext
--card-border	#E2DFFF (Hell-Grau-Lila)	#231E3D (Dunkel-Violett)	Ränder von Karten & Feldern

9.2 Typografie & Micro-Animations

- **Fraunces & Plus Jakarta Sans:** Fraunces (Serif) wird im Header für das Logo verwendet, um Intimität zu vermitteln. Plus Jakarta Sans dient dank breiter geometrischer Formen als Hauptschriftart für die UI.
- **Flame Wobble & Flicker (`.animate-flicker`):** Animiert die Flammen-Icons bei aktiven Antwortserien mit zufallsnahen Rotationsänderungen (± 3 Grad), um das Flackern einer echten Kerze nachzuahmen.

10. Datenbank-Struktur & Schema-Referenz

Die PostgreSQL-Datenbank strukturiert alle Relationen, Integritätsbedingungen (Foreign Keys) und Trigger.

10.1 Profile-Tabelle (public.profiles) mit neuen Spalten

```
CREATE TABLE public.profiles (  
  id uuid PRIMARY KEY REFERENCES auth.users(id) ON DELETE CASCADE,  
  display_name text,  
  partner_id uuid REFERENCES public.profiles(id) ON DELETE SET NULL,  
  partner_code text UNIQUE,  
  avatar_url text,  
  intro_completed boolean DEFAULT false NOT NULL,  
  partner_since timestamptz,  
  last_answer_reset_at timestamptz,  
  nudge_cooldown timestamptz,  
  nudge_count integer DEFAULT 0 NOT NULL,  
  last_nudge_at timestamptz,  
  total_answers integer DEFAULT 0 NOT NULL,  
  total_matches integer DEFAULT 0 NOT NULL,  
  nudges_sent integer DEFAULT 0 NOT NULL,  
  morning_answers_count integer DEFAULT 0 NOT NULL,  
  night_answers_count integer DEFAULT 0 NOT NULL,  
  lunch_answers_count integer DEFAULT 0 NOT NULL,  
  last_minute_answers_count integer DEFAULT 0 NOT NULL,  
  long_answers_count integer DEFAULT 0 NOT NULL,  
  both_long_answers_count integer DEFAULT 0 NOT NULL,  
  journal_views integer DEFAULT 0 NOT NULL,  
  streaks_rebuilt integer DEFAULT 0 NOT NULL,  
  avatar_change_count integer DEFAULT 0 NOT NULL,  
  time_sync_5min_count integer DEFAULT 0 NOT NULL,  
  time_sync_1min_count integer DEFAULT 0 NOT NULL,  
  perfect_rankings_count integer DEFAULT 0 NOT NULL,  
  perfect_match_days_count integer DEFAULT 0 NOT NULL,  
  perfect_tot_days_count integer DEFAULT 0 NOT NULL,  
  answered_valentines boolean DEFAULT false NOT NULL,  
  answered_new_years boolean DEFAULT false NOT NULL,  
  answered_christmas boolean DEFAULT false NOT NULL,  
  answered_halloween boolean DEFAULT false NOT NULL,  
  answered_labor_day boolean DEFAULT false NOT NULL,  
  answered_unity_day boolean DEFAULT false NOT NULL,  
  answered_anniversary boolean DEFAULT false NOT NULL,  
  answered_leap_day boolean DEFAULT false NOT NULL  
);
```

10.2 Antworten-Tabelle (public.answers)

```
CREATE TABLE public.answers (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,  
  day_key date NOT NULL,  
  choice text NOT NULL,  
  created_at timestamptz DEFAULT timezone('utc'::text, now()) NOT NULL,  
  CONSTRAINT answers_user_id_day_key_key UNIQUE (user_id, day_key)  
);
```

10.3 Streaks-Tabelle (public.streaks)

```
CREATE TABLE public.streaks (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,  
  partner_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE,  
  current_streak integer DEFAULT 0 NOT NULL,  
  longest_streak integer DEFAULT 0 NOT NULL,  
  last_answer_date date,  
  streak_history jsonb DEFAULT '[]'::jsonb NOT NULL,  
  freeze_history jsonb DEFAULT '[]'::jsonb NOT NULL,  
  CONSTRAINT streaks_user_id_partner_id_key UNIQUE (user_id, partner_id)  
);
```

10.4 Zusätzliche Tabellen-Spezifikationen & failed_generations

```
-- failed_generations (Retry-Queue)
CREATE TABLE public.failed_generations (
  day_key date PRIMARY KEY,
  failed_at timestamptz DEFAULT now() NOT NULL,
  retry_count integer DEFAULT 0 NOT NULL
);

-- Ankündigungen (Server-Driven Popups)
CREATE TABLE public.announcements (
  id bigint PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  version_code integer UNIQUE NOT NULL,
  title text NOT NULL,
  content text NOT NULL,
  type text DEFAULT 'info'::text NOT NULL,
  emoji text DEFAULT '■'::text NOT NULL,
  button_label text DEFAULT 'Los geht's'::text NOT NULL,
  action_route text,
  created_at timestamptz DEFAULT now() NOT NULL
);

-- Ankündigung-Views (Gelesen-Status)
CREATE TABLE public.announcement_views (
  id bigint PRIMARY KEY GENERATED BY DEFAULT AS IDENTITY,
  user_id uuid REFERENCES public.profiles(id) ON DELETE CASCADE NOT NULL,
  announcement_id bigint REFERENCES public.announcements(id) ON DELETE CASCADE NOT NULL,
  seen_at timestamptz DEFAULT now() NOT NULL,
  CONSTRAINT announcement_views_user_id_announcement_id_key UNIQUE (user_id, announcement_id)
);

-- Push-Abonnements
CREATE TABLE public.push_subscriptions (
  user_id uuid PRIMARY KEY REFERENCES public.profiles(id) ON DELETE CASCADE,
  subscription jsonb NOT NULL,
  created_at timestamptz DEFAULT now() NOT NULL
);
```

11. PL/pgSQL Funktionen, Trigger & RPCs

11.1 Partnerkopplung: `public.link_partners()`

Führt eine atomare, bidirektionale Verknüpfung zweier Profile durch. Die Funktion nimmt den ``partner_code_to_link`` entgegen, sucht die zugehörige UUID und setzt gegenseitig ``partner_id`` sowie ``partner_since`` auf den aktuellen Zeitstempel.

11.2 Kontolöschung: `public.delete_user_account()`

Löscht das Konto des angemeldeten Benutzers sicher aus ``auth.users``. Um eine Verletzung von Fremdschlüsselbedingungen zu vermeiden, hebt die Funktion zuerst die Koppelung beim Partner auf (setzt dort ``partner_id`` auf NULL) und löscht anschließend den User-Datensatz.

11.3 Streak-Verwaltung: `public.update_streak()`

Ein AFTER INSERT-Trigger auf `public.answers`. Wird eine Antwort gespeichert, ermittelt der Trigger das aktuelle Datum und prüft den Tag der letzten Antwort. War die letzte Antwort gestern, wird der Streak inkrementiert und das Datum an die `streak_history` angehängt. Liegt die letzte Antwort länger zurück, wird die Serie auf 1 zurückgesetzt.

Historie-Schutz: Wenn der Streak zurückgesetzt wird, wird die Historie nicht überschrieben, sondern der heutige Tag wird angehängt. Dies stellt sicher, dass die vollständige Antwort-Historie in `streak_history` erhalten bleibt.

11.4 Antwort-Reset: `public.reset_today_answers(day_key_param text)`

Erlaubt es Nutzern, ihre heutigen Antworten zu löschen, um sie neu zu beantworten. Die Funktion implementiert einen harten 7-Tage-Cooldown. Sie prüft ``profiles.last_answer_reset_at``. Liegt der Wert weniger als 7 Tage in der Vergangenheit, wird die Ausführung mit einer Fehlermeldung (RAISE EXCEPTION) abgebrochen.

11.5 Cronjobs (Datenbereinigung & Retry-Queue)

- **`public.cleanup_old_answers()`:** Löscht täglich um 3:00 Uhr alle Antworten, die älter als 90 Tage sind.
- **`public.retry_failed_generations()`:** Wird stündlich ausgeführt. Sucht in ``failed_generations`` nach Fehlversuchen und sendet mittels der PostgreSQL-Erweiterung *pg_net* einen neuen asynchronen HTTP-POST-Request an die Edge Function zur Fragengenerierung.

11.6 Meilenstein-Freischaltung: `check_and_unlock_milestones()`

Ein vollautomatisches Daten-Auswertungssystem auf Datenbankebene. Um die historische Unvollständigkeit der Tabelle `answers` zu umgehen, erfasst das System alle Fortschritte über persistente Zähler-Felder in der Tabelle `profiles`. Diese Felder werden automatisch über verschiedene Trigger gewartet (inklusive der neuen Spalten für Feiertage).

12. Web-Push Notifications & Edge Function Integration

Die PWA-Push-Infrastruktur nutzt Deno-basierte Supabase Edge Functions für kryptografisch gesicherte Benachrichtigungen.

12.1 Der Push-Notification-Dienst (send-push-notification)

Dieser Deno-Dienst empfängt API-Requests vom Client und bereitet das Payload-JSON vor. Die Übertragung erfolgt über das Web-Push-Protokoll direkt an den Push-Service des Empfänger-Browsers.

12.2 Kryptografische Absicherung (VAPID & AES-128-GCM)

Die Push-Verschlüsselung wird vollständig über die Deno-interne Web Crypto API realisiert:

1. **VAPID-Signierung (ES256):** Ein JWT mit der Ziel-Audienz und Ablaufzeit von 12 Stunden wird mit dem privaten VAPID-Schlüssel (ECDSA P-256) signiert.
2. **Schlüsselaustausch (ECDH):** Deno generiert ein temporäres ECDH-Schlüsselpaar und berechnet über Diffie-Hellman-Schlüsselaustausch mit dem Client ein Shared Secret.
3. **Schlüsselableitung (HKDF):** Das System leitet den symmetrischen Content Encryption Key (CEK) und ein Nonce ab.
4. **Verschlüsselung (AES-128-GCM):** Die Payload wird verschlüsselt und als binärer Datenstrom gesendet.

13. Systemdatenfluss & Datenarchitektur

Nachfolgend wird der typische Ablauf einer Interaktion visuell dargestellt, um das Zusammenspiel zwischen Frontend-Aktionen, Trigger-Prozessen und Hintergrund-Diensten zu verdeutlichen.

Phase	Akteur	Aktion & Systemreaktion
1. Start	Client App.tsx	Prüft Session. Lädt Profildaten. Fragt nach heutigen Fragen in daily_questions. Trigger-Prefetching für morgen.
2. Beantwortung	Client Questions.tsx	Nutzer beantwortet 3-4 Fragen. UI führt optische Verschlüsselungs-Animation aus und speichert Daten via Supabase Client.
3. Persistierung	Postgres DB	Speichert Zeile in public.answers. Löst Trigger 'on_answer_update_streak' aus, welcher Streaks & History aktualisiert.
4. Notification	Edge Function	Questions.tsx ruft 'send-push-notification' auf. VAPID-verschlüsselte Push-Nachricht wird an Partner gesendet.
5. Auswertung	Edge Function	Bei Aufruf von StatsModal.tsx berechnet 'calculate-stats' über gte-small Embeddings & Rangabstände die Beziehungsdaten.

13.1 Fazit

Dank des Server-Driven UI-Prinzips, der server-seitigen Trigger für geschäftskritische Zustände (Streaks, Koppelungen) und der Auslagerung komplexer mathematischer Modelle in isolierte, bedarfsgesteuerte Edge Functions ist die Bisou App extrem ausfallsicher, performant und leicht erweiterbar. Alle Kernkomponenten arbeiten autark und greifen über sichere RLS-Policing-Mechanismen ineinander.